

VORBEREITUNG ZUR VORLESUNG

Handout 07 – Der komplette Stack im Überblick

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Architektur einer Mehr-Container-Anwendung
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Alles zusammengeführt

Der Capstone verbindet jedes Einzelthema des Kurses zu einem funktionierenden System. Nichts davon ist neu – Sie setzen nur zusammen, was Sie schon können:

Baustein	woher
eigenes Image aus einem Dockerfile	Übung 03
Datenbank + Netzwerk (Service Discovery)	Übung 04
deklarativer Compose-Stack + Healthcheck	Übung 05
Password als Secret	Übung 06
persistente Daten im Volume	Übung 02

Die Architektur

Drei Container in einem von Compose angelegten Netzwerk:

- **db** (PostgreSQL) – speichert die Logeinträge; Daten im Volume `pgdaten`, Passwort aus einem Secret, mit Healthcheck.
- **adminer** – Web-Oberfläche zur Kontrolle der Daten (Port 8080).
- **log_tool** – unser Python-Werkzeug; verbindet sich über den Netzwerknamen `db`, liest Konfiguration aus Umgebungsvariablen und das Passwort aus dem Secret.

Darüber wacht **Portainer** (aus der Dev-Container-Einrichtung), in dem Sie Container, Netzwerk und Volume sehen.

Das Muster

Anwendung + Datenbank, sauber in Container verpackt, mit einem Befehl startbar: Genau dieses Grundmuster begegnet Ihnen in fast jedem realen Projekt wieder.

Warum das die Vorbereitung auf die Python-Kurse ist

In den kommenden Python-Modulen schreiben Sie Anwendungen, die Daten speichern und mit anderen Diensten sprechen. Dank dieses Kurses können Sie eine solche Anwendung **reproduzierbar** ver-

packen: Ihre Kommilitonen und Ihre Prüferin starten sie mit einem einzigen `docker compose up` – ohne „bei mir läuft es aber“. Der Umstieg von einer Datei-basierten Speicherung (Übung 02) auf eine echte Datenbank (ab Übung 04) zeigt außerdem, wie eine Anwendung wächst, ohne dass sich ihr Aufruf ändert.

Reproduzierbarkeit als roter Faden

Der eigentliche Gewinn von Docker ist nicht „Container“, sondern **Reproduzierbarkeit**: Das selbe `compose.yaml` erzeugt überall denselben Stack. Die Version der Datenbank steht fest (`postgres:17`), die Abhängigkeiten Ihres Codes stehen in `requirements.txt`, die Startreihenfolge regelt der Healthcheck. Damit ist Ihr System nicht nur lauffähig, sondern **wieder** lauffähig – auf jedem Rechner, zu jeder Zeit.

Wenn etwas klemmt

- „connection refused“ gleich nach `up`: Der Healthcheck sollte das verhindern; sonst den `log_tool`-Aufruf einmal wiederholen.
- Änderungen am Code wirken nicht: `docker compose up -d --build` erzwingt den Neubau des `log_tool`-Images.
- Daten unerwartet weg: Haben Sie `down -v` benutzt? Das löscht das Volume mit.